

Multi-Core Global Scheduling —

Milestone 2: Scientific Contextualization and Tradeoff Analysis

Md Mostafizur Rahman

Electronic Engineering, Hochschule Hamm-Lippstadt (HSHL)
Lippstadt, Germany md-mostafizur.rahman@stud.hshl.de

Abstract—Structured context and tradeoff analysis (M2) for Gracioli et al. (2013). Goal: locate the paper in the multicore real-time scheduling landscape, compare neighbouring paradigms and tools, and make the engineering tradeoffs explicit — including where my own transfer study (general-purpose OS measurement + UPPAAL verification) sits. Notes are structured, not publication prose.

I. SCIENTIFIC CONTEXT AND POSITIONING

- The paper sits between *two established lines* and connects them:
 - scheduling theory (Bertogna, Baruah, Baker, Goossens) — the schedulability tests;
 - Linux-based overhead measurement (Brandenburg, Bastoni, Lelli) — the empirical method.
- Its novelty is the substrate: prior overhead studies all ran on Linux RT patches; Gracioli et al. provide the first purpose-built-RTOS baseline (EPOS) and argue the OS substrate is a confound in every earlier comparison.
- **Density of related work:**
 - *Strong* — global/partitioned scheduling theory and Linux overhead measurement (paper extends both).
 - *Sparse* — purpose-built open-source RTOS for multicore HRT (EPOS is the first example; no comparable system).
 - *Bounded out of scope* — cache-aware/clustered scheduling, mixed-criticality, DRAM interference, ARM embedded multicore.

II. RELATED SCHEDULING PARADIGMS

A. Global scheduling

- **G-EDF** — dynamic priority by absolute deadline; suboptimal for HRT but tractable overhead; 8 sufficient tests, all pessimistic (worst for $u_i > 0.5$).
- **Proportional-fair (PFair/DP-Fair/QPS)** — optimal (schedules any feasible set), but preemptions/migrations at every quantum make overhead dominate; practically infeasible for embedded HRT $\Rightarrow$ excluded.
- Shared assumption: any core can run any task at any instant — needs shared memory, cache coherence, a global lock (or lock-free queue), and IPIs for remote rescheduling.

B. Partitioned scheduling

- **P-EDF** — FFD/BFD/WFD packing + per-core EDF; tighter test (Liu & Layland per core); no migration $\Rightarrow$ no IPI, no migration CPMD.

TABLE I
MULTICORE REAL-TIME SCHEDULING PARADIGMS

Paradigm	Idea	Status in paper
G-EDF (subject)	one global queue; m earliest deadlines run; migration	measured on EPOS & LITMUS ^{RT}
P-EDF (baseline)	offline bin-packing + per-core EDF	comparison target
PFair / DP-Fair / QPS	proportional-fair, optimal on m cores	excluded: per-quantum preempt/migrate overhead
T-L plane (Cho 2006)	time / remaining-execution abstraction	not compared (distant from EDF)
Clustered EDF (C-EDF)	G-EDF inside cache-aligned clusters	not in EPOS; future work
Semi-partitioned	pin most tasks, split a few	not in EPOS; future work

- Limitation: bin-packing is NP-hard and fails on heavy tasks ($5 \times u_i = 0.51$ do not fit 4 cores though $U = 2.55$).
- Variants: task splitting (Kato & Yamasaki; Burns) recovers lost utilisation; constant-time admission (Masrur) cuts packing cost.
- Aligns naturally with AUTOSAR's static core-affinity model.

C. Clustered and semi-partitioned (the active frontier)

- **C-EDF** (Bastoni 2010b, the paper's closest related work): tasks clustered per shared cache level; G-EDF within a cluster. C-EDF-L2 beat *both* G-EDF and P-EDF on a 24-core Intel platform by limiting cross-cluster CPMD. Not implemented in EPOS $\Rightarrow$ a genuine gap vs the state of the art: the paper's two-way (global vs partitioned) framing omits the paradigm already shown to win.
- **Semi-partitioned**: recovers P-EDF's packing losses without full G-EDF IPI/lock cost; more complex admission control; also left as EPOS future work.

III. RELATED TOOLS AND IMPLEMENTATIONS

- **EPOS** — component-based, object-oriented RTOS; compile-time criterion/queue specialisation, inlined mediators, library mode, distributed alarm handler. Chosen because it is the only open-source RTOS supporting global

TABLE II
LINUX-BASED REAL-TIME EXTENSIONS VS EPOS

System	Approach	Limitation vs EPOS
SCHED_DL	EDF class in mainline Linux; CBS; cgroups	Linux non-determinism
Linux/RK	portable RK; callback hooks	callback overhead; latency
ExSched	external plugin framework	API call per scheduling event
AQuoSA	QoS patch; CBS/IRIS/GRUB	SRT-focused, not HRT
AIRS	on SCHED_DL; multimedia	SRT only

TABLE III
CORE TRADEOFFS (G-EDF VS P-EDF)

Axis	G-EDF	P-EDF
Queue / lock	one global, $O(n)$	per-core, $O(n/m)$
Migration / IPI	allowed / needed	none / none
Flexibility	dynamic load sharing	fixed at design time
Heavy tasks	handled naturally	can block packing
Weakness	CPMD, pessimistic tests	NP-hard allocation

RT scheduling at publication time; its component model makes adding G-EDF a direct code-reuse demonstration.

- **LITMUS^{RT}** — Linux RT patch with interchangeable plugins (G-EDF, P-EDF, C-EDF, PFair) and Feather-Trace logging (≈ 1 instruction / trace point). The standard platform for overhead studies $\Rightarrow$ the right foil: its Linux base is exactly the argued problem.
- Common assumption across all of these: the Linux kernel is an acceptable real-time substrate. Gracioli et al. challenge it directly — 2 ms context-switch outliers are Linux internals, and for HRT those are deadline misses, not noise.

A. Timing-analysis methods (positioning my UPPAAL angle)

- **Measurement-based** (the paper, and my transfer study): sampled “observed worst-case” overhead from many runs; realistic but not exhaustive; cannot certify rare tails.
- **Analytical WCET** (aiT, OTAWA): static bounds on program code; sound but pessimistic; a *different layer* of the timing stack, not directly comparable to sampled OS overhead.
- **Formal model checking** (UPPAAL / timed automata): exhaustive over all behaviours *admitted by the model*; only as realistic as its parameters. My report uses UPPAAL to turn measured values into a provable deadline consequence — a complement, not a substitute, for measurement.

IV. TRADEOFF ANALYSIS

- **Flexibility vs overhead.** G-EDF balances load dynamically and absorbs heavy tasks, paying a global lock, IPIs, and

migration CPMD. P-EDF removes those costs but moves flexibility to design time; if packing fails the system is unschedulable even below capacity.

- **Optimality vs implementability.** PFair is optimal but its per-quantum migrations make it undeployable in embedded HRT; G-EDF is suboptimal (sufficient tests) yet deployable. The real frontier is tightening G-EDF’s tests, not chasing optimality.
- **Precision vs runtime (of the test itself).** The most precise G-EDF test (Baruah 2007) is pseudo-polynomial and was excluded for light uniform loads on cost grounds. Schedulability analysis is itself a resource problem — online admission cannot afford pseudo-polynomial tests.
- **Migration vs cache locality.** Each G-EDF migration risks evicting the working set (CPMD), inflating e'_i . Measured: for WSS ≤ 128 KB P-EDF is clearly better; at 512 KB the gap narrows; at ≥ 512 KB LLC thrash dominates and P-EDF $\approx$ G-EDF.
- **Formal guarantees vs deployability.** EPOS library mode gives lowest overhead but no isolation/certification/tooling; LITMUS^{RT} gives Linux tooling and isolation but inherits non-determinism. Neither is certifiable (ISO 26262 ASIL-D / DO-178C); industrial RTOSes (VxWorks, QNX) certify but lack global multicore scheduling. A research-production gap remains.
- **Measurement vs formal verification (my report’s axis).** Measurement yields realistic parameters but cannot cover every path; timed automata cover every modelled path but only as realistically as their inputs. The two are complementary — the basis for combining perf-measured WCET with UPPAAL in M3.

V. CONSEQUENCES FOR EMBEDDED REAL-TIME SYSTEMS

- **Timing / determinism.** At short periods (3–33 ms), LITMUS^{RT} overhead can consume a light task’s entire slack. EPOS tick overhead is thread-count-independent — a determinism property absent from Linux-based systems. G-EDF IPI jitter must be absorbed into e'_i .
- **Memory.** G-EDF: one queue of size n + one lock; P-EDF: m queues + m locks (same memory, lower contention). Library mode drops MMU cost but also fault isolation. WSS exceeding shared L3 (8 MB) thrashes and equalises the two.
- **Energy** (not measured, inferable). Each G-EDF migration forces a cache refill from DRAM ($\sim 10\times$ L3 energy); P-EDF’s static binding reduces migrations — but the advantage vanishes at large WSS. Periodic ticks also block deep sleep on idle cores.
- **Deployment.** Automotive: P-EDF’s static partitioning matches AUTOSAR core-affinity; G-EDF migration is harder under ASIL decomposition. IoT / high-frequency control: short periods, light load, small n — overhead-to-period ratio dominates and EPOS G-EDF wins. Deeply embedded (no MMU): library mode is natural, but a shared

bus (not QPI) would raise IPI latency above the paper’s numbers.

VI. WHY RELATED WORK IS SPARSE OR INDIRECT

- **Purpose-built RTOS for multicore HRT — sparse.** Industrial RTOSes are proprietary, OSEK-uniprocessor-based, or publish no peer-reviewed overhead data; comparison needs source access.
- **Cache-aware / clustered — parallel.** C-EDF and FPCA address a different question (exploit cache topology); comparing would require EPOS implementations (future work).
- **Mixed-criticality — different model.** Vestal-style multiple-WCET, mode-switch semantics change the test structure entirely; out of scope, not an oversight.
- **DRAM interference — parallel thread.** Acknowledged as an overhead source but needs bandwidth-bound workloads and DRAM instrumentation.
- **Analytical WCET tools — methodological mismatch.** aiT/OTAWA produce analytical code bounds; this paper uses sampled OS overhead — different layers.
- **ARM embedded multicore — platform gap.** Different coherence, IPI mechanism, and in-order pipeline would change every measured value; listed as future work.

VII. POSITIONING OF MY TRANSFER STUDY

- I substitute an *accessible* substrate contrast (Windows 11 vs Ubuntu on one AMD Ryzen 7 7435HS) for the EPOS vs LITMUS^{RT} contrast, to test whether “OS substrate changes effective WCET” reproduces *qualitatively* on commodity hardware.
- I add a UPPAAL timed-automata model to turn the measured values into a formally verified deadline consequence — placing the work at the intersection of the overhead-measurement line and the formal-methods line.
- Explicit scoping: this is a qualitative analogue, *not* an EPOS–LITMUS^{RT} reproduction (general-purpose OSes are neither RTOS); it isolates no single overhead source. This honest boundary is developed and critiqued in M3.